Fitt4Sences

Executive Summary of the Final Degree Project

Native Android application for comprehensive personal wellness tracking Kotlin - Jetpack Compose - MVVM - Supabase - PostgreSQL

Document prepared as a professional summary for GitHub, portfolio use and technical presentation.

1. Executive summary

Fitt4Sences is a native Android application focused on comprehensive personal wellness tracking. The project integrates four main health pillars into a single tool: nutrition, physical exercise, hydration and body metrics. Its technical value is not limited to data entry; it correlates the data to provide a coherent daily overview of the user's physical state.

Key idea: the app does not place critical calculations only in the mobile interface. The PostgreSQL database acts as the source of truth and calculates BMR, BMI, burned calories, hydration data and daily energy balance.

65	9	26	13	48
Kotlin files	screens	UI components	DB tables	RLS policies

2. Problem addressed

The project responds to a clear need: avoiding the usual fragmentation of health applications. Many solutions focus on a single area and force the user to manually combine nutrition, exercise, hydration and body metrics.

Identified problem	Impact on the user	Fitt4Sences response
Isolated data	The user does not get a complete view of their daily status.	It unifies nutrition, exercise, hydration and metrics in a central dashboard.
Inconsistent calculations	If calculations live only in the client, they may vary between devices or sessions.	Critical calculations are delegated to PostgreSQL through functions, triggers and generated columns.
Lack of real privacy	Health-related data requires per-user isolation.	Row Level Security in Supabase/PostgreSQL ensures each user only accesses their own data.

3. Main objectives

- Design and develop a functional Android app to manage personal wellness from a single platform.
- Implement secure authentication with registration, email confirmation, password recovery and persistent sessions.
- Build operational modules for nutrition, exercise, hydration and physical metrics.
- Apply MVVM architecture with repositories and reactive state management through StateFlow.
- Integrate Supabase as the backend and PostgreSQL as an active business logic engine.
- Ensure consistency, security and traceability through RLS, constraints, functions and triggers.

4. Technical architecture

The solution is organized into three main layers with clear separation of responsibilities. The interface captures events and renders state; ViewModels coordinate the flow; repositories encapsulate backend access; and PostgreSQL executes critical business logic.

Layer	Responsibility	Technologies / elements
Presentation	Render the UI, capture user actions and observe reactive state.	Jetpack Compose, Material Design 3, Navigation Compose, StateFlow.
Data	Encapsulate queries, CRUD operations and RPC invocations.	Kotlin repositories, Supabase Kotlin SDK, PostgREST, Ktor, kotlinx.serialization.
Backend	Persistence, calculations, security, integrity and business rules.	Supabase Auth, PostgreSQL 17, SQL functions, triggers, RLS, enums, CHECK constraints.

Main flow: View/Composable -> ViewModel -> Repository -> Supabase/PostgREST/RPC -> PostgreSQL -> Repository -> ViewModel -> StateFlow -> Compose.

5. Functional modules

Module	Status	Key features
Authentication	Complete	Login, registration, email confirmation, password recovery through deeplink and persistent session.
Dashboard	Complete	Central summary with user data, BMI, BMR, physical metrics and daily energy balance.
Nutrition	Complete	Meal registration and deletion by type, calorie total and automatic daily balance recalculation.
Exercise	Complete	Default habits, duration-based sessions, daily accumulation, MET values, burned calories, achievements and weekly charts.
Hydration	Complete	Water glass tracking, daily target, configurable ml per glass and weekly evolution.
Physical metrics	Complete	Weight, height, age, sex, automatic BMI and BMR calculated from the database.
Statistics	Partial	Currently focused on hydration and exercise; prepared for future global analytics.

6. Technology stack

Area	Technology	Use in the project
Language	Kotlin	Main language of the Android application.
UI	Jetpack Compose + Material 3	Modern, declarative and reusable interfaces.
Architecture	MVVM + StateFlow	Separation of responsibilities and reactive state.
Backend	Supabase	Authentication, PostgREST, RPC and cloud management.
Database	PostgreSQL 17	Relational persistence and server-side business logic.
HTTP / API	Ktor + Supabase SDK	Asynchronous communication with the backend.
Charts	Vico Charts	Weekly activity and hydration visualization.

Area	Technology	Use in the project
Build	Gradle Kotlin DSL	Project configuration, dependencies and BuildConfig.

7. Database and business logic

The database is one of the most important parts of the project. It does not only store records: it calculates, validates, connects and protects data.

Element	Quantity / example	Role
Tables	13 tables	Users, physical metrics, daily nutrition, meals, habits, sessions, hydration, exercise settings and future tables.
SQL / RPC functions	16 functions	BMR calculation, metrics upsert, balance recalculation, exercise session registration and hydration.
Triggers	16 triggers	Burned calories, automatic timestamps and authenticated user assignment.
RLS policies	48 policies	Data isolation per authenticated user.
Constraints	12 CHECK + FK + enums	Prevent invalid data such as negative calories, null MET values or out-of-range water glasses.

8. Security

- Authentication with Supabase Auth and JWT tokens.
- Row Level Security on operational tables to isolate data by user.
- Credentials managed through local.properties and BuildConfig, avoiding literal values in the source code.
- SECURITY DEFINER functions for operations that need to query multiple tables without breaking the RLS model.
- Double validation: client-side through ViewModels and server-side through CHECK, FK, ENUM and policies.

9. Testing and validation

Validation was mainly performed through manual functional tests and direct Supabase queries. Although dependencies for JUnit, Espresso and Compose UI Test are configured, automated testing remains a future improvement.

Tested area	Validation performed	Expected result
Authentication	Registration, confirmation, login, recovery and deeplinks.	Secure access and correct navigation.
Nutrition	Meal insertion/deletion and balance recalculation.	Daily balance updated after each operation.
Exercise	Session registration, accumulation and MET calculation.	Burned calories and statistics updated.

Tested area	Validation performed	Expected result
Hydration	Glass tracking, settings and weekly evolution.	Correct daily progress and weekly chart.
Security	Attempted access to another user's data.	PostgREST returns an empty result set due to RLS.

10. Final results

The result is a functional MVP installable on Android API 24 or higher, with the main modules operational and a database prepared for future product evolution.

Indicator	Result
Product	Functional MVP with authentication, dashboard, nutrition, exercise, hydration and physical metrics.
Codebase	65 Kotlin files, 9 screens, 26 reusable components, 9 ViewModels and 9 repositories.
Backend	PostgreSQL 17 on Supabase with 13 tables, 16 functions, 16 triggers and 48 RLS policies.
Skills demonstrated	Android development, declarative UI, MVVM architecture, data access, security, SQL, documentation and troubleshooting.

11. Known limitations

- The application is Android-only; there is no iOS or web client in the MVP.
- Calorie logging is manual and does not yet use the food catalog prepared in the database.
- There is no offline mode or deferred synchronization.
- Push notifications for reminders are not implemented.
- Testing is manual; unit, integration and automated UI tests are still missing.
- Availability depends on the Supabase cloud service.

12. Future improvements

- Connect the food catalog and comidas_alimentos table to the nutrition flow.
- Integrate AI chat and personalized recommendations using the prepared tables.
- Create global weekly statistics that consolidate all modules.
- Add push notifications for hydration, meals and exercise.
- Implement automated tests for ViewModels, repositories and Compose screens.
- Add offline mode with Room or DataStore and later synchronization with Supabase.
- Prepare future web or iOS clients consuming the same PostgREST backend.

13. Recommended GitHub usage

For GitHub, this summary should be placed inside the docs folder next to the full final report. The main README should be shorter and portfolio-oriented: product description, stack, screenshots, architecture, features and documentation links.

Recommended file	Use
README.md	Quick project presentation for recruiters, companies and evaluators.
README_ES.md	Spanish README version if a bilingual repository is desired.
docs/TFG_RESUMEN_ES.pdf	Executive summary in Spanish for quick review.
docs/TFG_SUMMARY_EN.pdf	Equivalent English copy for an international portfolio.
docs/TFG_Fitt4Sences.pdf	Full final report, only after verifying that it contains no sensitive data.

14. Conclusion

Fitt4Sences is a strong Final Degree Project because it combines a modern Android application with a real and secure relational backend. The most remarkable decision is using PostgreSQL as an active business logic engine, which raises the project above a simple data-entry app. As a portfolio piece, it demonstrates architectural judgment, security, backend integration and the ability to document a complete system.

Source used: Fitt4Sences final degree project report provided by the author. This document summarizes the essential points and does not replace the full report.